

EXAMEN DE SEGURIDAD EN EL DISEÑO DE SOFTWARE

PRUEBA FINAL DE LA ASIGNATURA

Nombre y apellidos: _____ Grupo: _____ Fecha: _____

DNI: _____

1. ¿Qué es el principio de mínimo privilegio y por qué es importante en el diseño de software? (0,5 puntos)

El principio de mínimo privilegio establece que cada componente, usuario o proceso del sistema debe tener únicamente los permisos estrictamente necesarios para realizar su función, y ninguno más. Es importante porque limita el daño potencial ante un compromiso de seguridad: si un atacante toma control de un componente con privilegios reducidos, el impacto queda contenido y no puede propagarse al resto del sistema.

2. Explica qué es una inyección SQL, cómo puede explotarse y qué medidas de diseño se deben aplicar para prevenirla. (2 puntos)

Una inyección SQL es un ataque en el que un atacante inserta o "inyecta" código SQL malicioso en una entrada de la aplicación que posteriormente se ejecuta en la base de datos sin validación. Por ejemplo, si un formulario de login construye la consulta concatenando directamente la entrada del usuario, un atacante puede introducir ' OR '1'='1 para saltarse la autenticación o DROP TABLE users; - para eliminar tablas.

Las medidas de diseño para prevenirla son: uso de consultas parametrizadas o prepared statements (nunca concatenar entrada de usuario en SQL), uso de ORMs que abstraigan la capa de base de datos, validación y sanitización estricta de todas las entradas, y principio de mínimo privilegio en el usuario de base de datos de la aplicación.

3. ¿Qué diferencia hay entre autenticación y autorización? (0,5 puntos)

La autenticación es el proceso de verificar la identidad de un usuario o sistema (¿quién eres?). La autorización determina qué acciones o recursos tiene permitido usar una identidad ya autenticada (¿qué puedes hacer?). Primero se autentica y luego se autoriza.

4. ¿Qué es el modelado de amenazas (threat modeling) y en qué fase del desarrollo debe realizarse? (1 punto)

El modelado de amenazas es un proceso estructurado para identificar, clasificar y priorizar las amenazas potenciales a un sistema antes de que este sea implementado. Consiste en analizar la arquitectura del sistema, identificar los activos a proteger, los posibles atacantes y los vectores de ataque, y definir contramedidas. Debe realizarse en la fase de diseño, antes de comenzar a escribir código, porque es el momento en que los cambios son menos costosos y se puede influir en las decisiones arquitectónicas fundamentales.

5. Describe qué es la defensa en profundidad (defense in depth) como estrategia de seguridad y pon al menos tres ejemplos de capas que se pueden implementar en una aplicación web. (2 puntos)

La defensa en profundidad es una estrategia de seguridad que consiste en implementar múltiples capas de controles de seguridad independientes, de modo que si una capa falla, las demás sigan protegiendo el sistema. El principio es que ningún control único es infalible.

Ejemplos de capas en una aplicación web:- Capa de red: firewall, WAF (Web Application Firewall), segmentación de red- Capa de transporte: HTTPS/TLS para cifrado en tránsito- Capa de aplicación: validación de entradas, autenticación y autorización robusta, gestión segura de sesiones- Capa de datos: cifrado en reposo, acceso con mínimo privilegio a la base de datos- Capa de monitorización: logging, alertas ante comportamiento anómalo, SIEM

6. ¿Qué es el OWASP Top 10 y para qué sirve en el contexto del diseño seguro? (1 punto)

El OWASP Top 10 es un documento de referencia elaborado por la Open Worldwide Application Security Project que recoge los diez riesgos de seguridad más críticos y frecuentes en aplicaciones web. En el contexto del diseño seguro sirve como guía de referencia para que desarrolladores y arquitectos conozcan las vulnerabilidades más comunes (inyección, broken authentication, XSS, etc.) y las tengan en cuenta desde las primeras fases del desarrollo, evitando introducirlas por desconocimiento.

7. Explica qué es un ataque de tipo Cross-Site Scripting (XSS), cuál es su impacto potencial y qué técnicas de diseño y desarrollo se deben emplear para mitigarlo. (2 puntos)

Un ataque de Cross-Site Scripting (XSS) consiste en inyectar código JavaScript malicioso en páginas web que posteriormente es ejecutado por el navegador de otros usuarios. Existen tres tipos principales: reflejado (el script viene en la petición), almacenado (el script se guarda en la base de datos) y basado en DOM.

El impacto potencial incluye: robo de cookies de sesión, suplantación de identidad, redireccionamiento a sitios maliciosos, keylogging en el navegador de la víctima y distribución de malware. Las técnicas de mitigación son: escapado correcto de salida (output encoding) según el contexto (HTML, JavaScript, URL), uso de Content Security Policy (CSP) para restringir scripts no autorizados, validación y sanitización de entradas, uso de atributo HttpOnly en cookies para impedir acceso desde JavaScript, y empleo de frameworks modernos que aplican escapado automático por defecto.

8. ¿Qué es el cifrado en tránsito y en reposo, y por qué ambos son necesarios? (1 punto)

El cifrado en tránsito protege los datos mientras se transmiten entre dos puntos (por ejemplo, entre el navegador y el servidor), usando protocolos como TLS/HTTPS. El cifrado en reposo protege los datos cuando están almacenados, por ejemplo en bases de datos, discos o backups, usando algoritmos como AES-256. Ambos son necesarios porque cubren amenazas distintas: el cifrado en tránsito protege frente a interceptación en la red (man-in-the-middle), mientras que el cifrado en reposo protege frente a acceso físico o lógico no autorizado al almacenamiento. Un sistema que solo cifra en tránsito expone los datos si alguien accede a la base de datos; uno que solo cifra en reposo expone los datos si se intercepta la comunicación.